

LLM 预训练之数据混合

Data Mixture · 深度研究手册

从“固定配比”到“可预测的训练分布优化”：Sampling、Reweighting、Proxy、Regression、Scaling Law 与 Dynamic Schedule

研究快照：2026-08-13

核心资料：LLaMA · mT5 · UniMax · DoReMi · DoGE · Data Mixing Laws · RegMix · Two-Phase Pretraining · Mixture Scaling Laws

核心判断 Data Mixture 不是“各数据源占多少”的静态表格，而是训练分布控制器。真正的工程对象是：在固定模型、token 预算、目标能力与数据约束下，决定每个训练阶段从哪个域采多少、重复多少次、以什么顺序出现，并用受控预训练实验验证其边际效用。

阅读地图

本册将 Data Mixture 与 Cleaning / Quality Scoring / Dedup 分开：前者不决定文档“是否合格”，而是在一组已经可用的数据池上控制 exposure。一个成熟 mixture policy 至少包含 domain weight、repeat cap、quality bucket weight、language sampling、phase schedule 与 sampler semantics。

章节	核心问题	关键 Know-how
1. 定义	Mixture 到底控制什么？	把“数据比例”改写为 token exposure distribution
2. 基线	Natural / Uniform / Temperature 怎么选？	先建可解释 baseline，再谈自动优化
3. 重复	上采样是否等于增加数据？	区分 unique tokens 与 repeated tokens，显式记录 epochs
4. 多语言	如何救低资源语言又避免过拟合？	Temperature + repeat cap；mT5 与 UniMax 是两条经典路线
5. 静态实例	真实 LLM 如何配数据？	LLaMA 1：Web 为主，但 Books/Wikipedia 被多 epoch 暴露
6. DoReMi	能否不用下游任务自动定权重？	excess loss + Group DRO + proxy-to-large transfer
7. DoGE	怎样直接优化泛化？	bi-level optimization + inter-domain dependency
8. RegMix	能否把调配方变成监督学习？	many tiny runs → regression surrogate → search
9. Mixing Laws	性能能否作为 mixture 的函数预测？	small-scale fit → unseen mixture / larger-scale extrapolation
10. Curriculum	权重是否应该随训练阶段变化？	two-phase / annealing / continual-pretrain schedule
11. Data constraints	目标域很小怎么办？	generic data 作为 regularizer，重复次数显式进入 scaling law
12. Production	如何上线？	versioned policy + proxy lab + mixture registry + online observability

1. 数据混合的数学对象：训练时真正被控制的是 exposure

设有 K 个数据域 $D_1 \dots D_K$ 。最简单的静态 mixture 用权重向量 $\alpha \in \Delta^K$ 定义训练分布：

$$P\alpha = \sum_i \alpha_i \cdot \text{Unif}(D_i), \text{ 其中 } \alpha_i = \text{每次采样来自第 } i \text{ 个域的概率}$$

DoReMi 将 domain weights 明确定义为采样概率，并指出这些比例会显著改变 LM 的效果；The Pile 等公开语料的默认权重通常是启发式决定的。[6]

概念	常见误解	正确工程含义
Corpus share	磁盘/文档占比就是训练占比	必须先经过 tokenizer，最终要看 token share
Sampling weight	等于原始数据比例	可以上采样/下采样，与 corpus size 解耦
Epoch	只有整库训练才有意义	每个域都可定义 expected repetitions = sampled tokens / unique tokens
Mixture	一个固定向量	实际常是 $\alpha(t)$ ：随训练阶段、模型状态或预算变化
Quality	质量高就应该占比高	高质量、小而重复的数据可能边际收益快速衰减
Target utility	单个域验证 loss	需要多目标：通用能力、目标域、遗忘、语言公平、风险等

Know-why 数据混合是一个“分布设计”问题：模型每处理一个 token 就花一次 compute。Mixture 的本质，是决定

有限 compute 应该被哪些 token 分布消费。

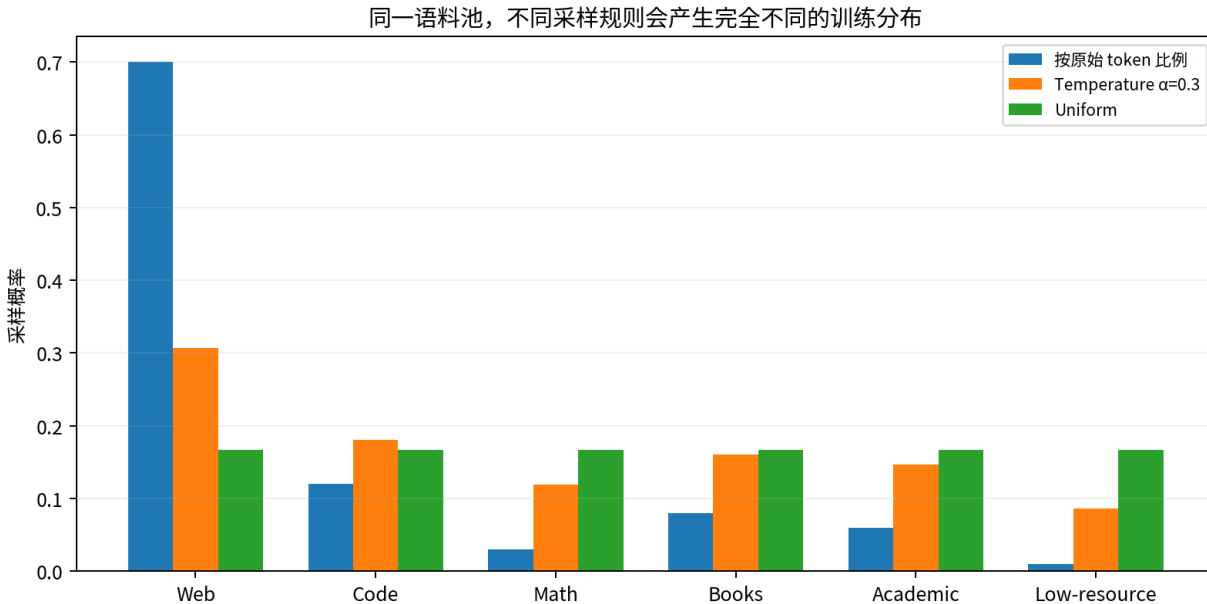


图 1 示例：原始 token 比例、temperature sampling 与 uniform 会产生完全不同的训练 exposure。

2. 最基本的四种混合策略

策略	公式/做法	优点	主要风险
Natural	$p_i = n_i / \sum n$	不制造重复；最接近原始数据分布	头部 Web 吞噬全部预算
Uniform	$p_i = 1/K$	每个域都有稳定 exposure	小域被高倍重复，容易过拟合
Temperature	$p_i \propto n_i^\alpha, 0 < \alpha < 1$	平滑长尾，易调、实现简单	α 是全局旋钮，无法表达复杂跨域互补
Quota / Cap	给域设 min/max tokens 或 max epochs	可控、适合治理与低资源保护	需要手工设阈值，可能浪费剩余预算

mT5 对 101 种语言使用 $p(L) \propto |L|^\alpha$ 的 temperature sampling，并选择 $\alpha=0.3$ ，作为高资源与低资源语言性能之间的折中；这种做法后来成为多语言 mixture 的经典基线。[3]

UniMax 指出纯 temperature sampling 会让尾部语言被反复采样；其核心改进是显式限制每种语言的重复次数，同时尽量均衡覆盖。论文报告 UniMax 在多种规模上优于标准 temperature sampling。[4]

Know-how 任何 mixture 实验都应该同时打印三列：sampling weight、实际 sampled tokens、effective epochs。只报“占比”会隐藏低资源域被重复几十次的事实。

3. 真实配方示例：LLaMA 1 说明“占比”和“重复次数”必须一起看

LLaMA 1 的公开训练配方包含 67% CommonCrawl、15% C4、4.5% GitHub、4.5% Wikipedia、4.5% Books、2.5% ArXiv、2% StackExchange。训练 1.4T token 时，Wikipedia 约 2.45 epochs、Books 约 2.23 epochs，而 GitHub 仅约 0.64 epoch。[2]

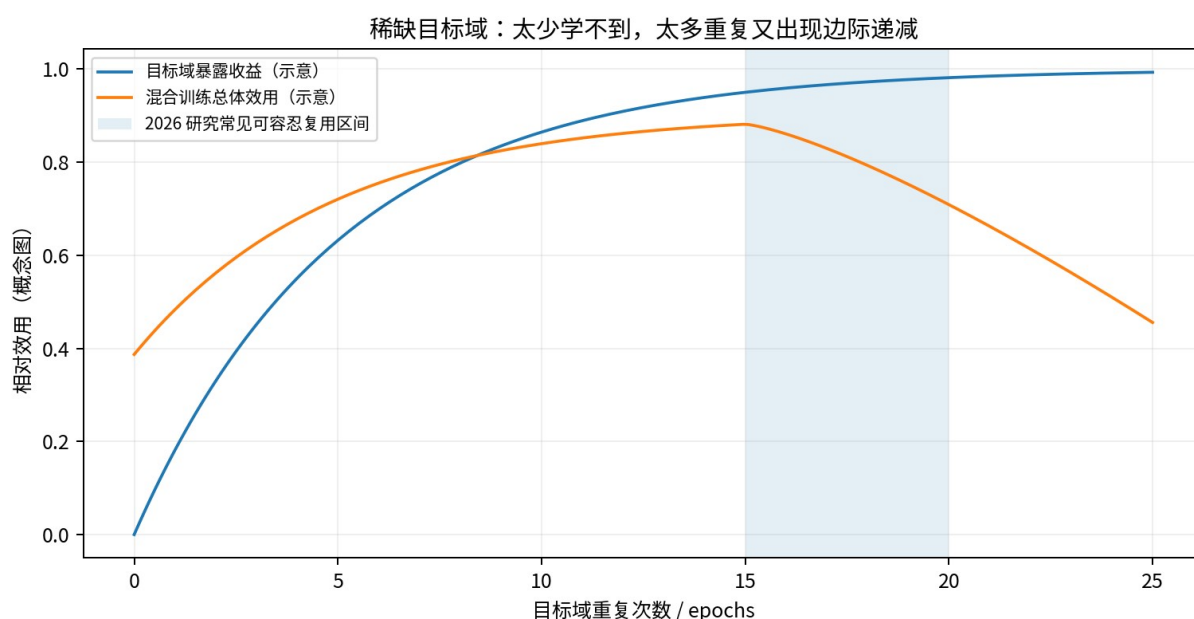
Domain	Sampling prop.	Approx. epochs (1.4T)	工程解读
CommonCrawl	67.0%	1.10	通用 Web 是容量主体
C4	15.0%	1.06	第二种 Web preprocessing 增加分布多样性
GitHub	4.5%	0.64	大语料不一定需要完整遍历
Wikipedia	4.5%	2.45	小而高密度数据被主动重复
Books	4.5%	2.23	长文本/叙事被多 epoch 暴露
ArXiv	2.5%	1.06	科学域按近 1 epoch 使用
StackExchange	2.0%	1.03	高质量问答近 1 epoch

关键洞察 “小域占比低”不等于“模型很少看到它”。当 corpus 本身小，4.5% 的持续采样可能意味着 2-3 个 epochs。Mixture policy 必须把 domain weight 与 corpus size 联合考虑。

4. 重复数据：上采样的收益为什么会逐渐衰减？

Scaling Data-Constrained Language Models 在最多 900B training tokens、9B 参数的实验中发现：数据受限时，约 4 epochs 以内的重复对 loss 的伤害可以很小，但继续重复后，额外 compute 的价值最终趋近于零。[5]

2026 年的 mixture-specific scaling 研究进一步在 2,000+ 次训练中发现，稀缺目标域与通用域混合时可容忍明显更多复用：典型最优重复可到 15-20 次，但最优值依赖目标数据量、compute budget 与模型规模。通用数据在这里起到 regularizer 作用。[12]



场景	推荐思考方式	不要做
大 Web 域	尽量 unique-token first	为了对齐比例而无意义重复
小高质量域	允许有上限的多 epoch	默认“高质量就无限复用”
低资源语言	sampling boost + repeat cap	只调 α 不看 epochs
Continual PT	目标域 + source replay	100% 新域导致遗忘
长 horizon	先预算 unique-token supply	把短实验最优配方无脑外推到 10T+

5. Mixture 的能力迁移：为什么直觉常常失效

域之间不是独立的“能力开关”。Web、Books、Code、Math、QA 会通过共享词汇、事实、语法、推理结构与表示产生正迁移或负干扰。Mixture 优化因此不是简单的“想要 code 就加 code”。

RegMix 的实验证明 data mixture 显著影响性能，而且域之间存在复杂交互；其结果中，Web corpora 与 downstream performance 的正相关甚至强于常被直觉认为“高质量”的 Wikipedia。[9]

DoReMi 的 The Pile 结果更极端：优化后的 Pile-CC 权重从约 11.2% 提升到约 60.6%，ArXiv 从约 10.5% 降到约 0.36%，PubMed Central 从约 10.7% 降到约 0.46%，但 8B 模型在所有域上的 perplexity 都得到改善。作者把这解释为不同 entropy 域的 sample efficiency 与跨域正迁移。[6]

Know-why 域权重不是“价值排名”。一个域被降权可能只是因为它更容易学、与其他域高度冗余，或可以通过别的域获得正迁移；一个看似噪声更高的 Web 域被升权，也可能因为它提供更广泛的表示基础。

6. DoReMi：用 Proxy + Excess Loss 自动找静态权重

DoReMi 的三步：①按参考 mixture 训练小 reference model；②训练同规模 proxy model，用 Group DRO 根据各域相对于 reference 的 excess loss 动态更新权重；③对训练轨迹中的权重取平均，用它重采样数据并训练大模型。其 280M→8B 实验中，优化权重的额外 compute 约为大模型训练的 8%。[6]

核心信号： $\text{excess loss} = L_{\text{proxy}}(x) - L_{\text{ref}}(x)$ 。高 excess loss 被解释为“reference 已能学会，但 proxy 还没有学到”的 headroom。

DoReMi 组件	作用	工程注意点
Reference mixture	定义基线难度	参考权重本身会影响 excess-loss 解释
Proxy model	低成本探索权重	需要验证 proxy→large 的 transfer
Group DRO	优先高 excess-loss 域	需要 smoothing，避免权重塌缩
Averaged weights	把动态轨迹压成静态配方	失去随训练阶段变化的信息
Large run	验证真实收益	最终仍必须 fixed-compute A/B

在 The Pile 上，DoReMi 的 8B 模型平均 few-shot 下游准确率比默认 mixture 高 6.5 个百分点，并以约 $2.6\times$ 更少训练步数达到 baseline accuracy。[6]

7. DoGE：把“权重优化”改写成泛化估计

DoGE 也是两阶段：先用 proxy model 的 bi-level optimization 学域权重，再按这些权重训练更大的 base model。与 DoReMi 不同，它强调 generalization estimation 与 inter-domain dependency；在 SlimPajama 上改善 perplexity 与 6 个 few-shot reasoning tasks，并能帮助 unseen OOD target domain。[7]

方法	优化信号	是否依赖目标域	优点	局限
DoReMi	per-domain excess loss / worst-case	不需要下游任务	目标无关、机制清晰	倾向静态最终权重
DoGE	generalization estimation / bilevel	可面向任意目标混合/OOD	显式建模跨域泛化	实现与计算更复杂
RegMix	small-run performance → regression	取决于训练标签指标	黑盒兼容、搜索灵活	需要大量小实验
Mixing Laws	mixture→loss 函数拟合	可定义目标 validation domains	能外推 unseen mixtures	函数假设/尺度转移需验证

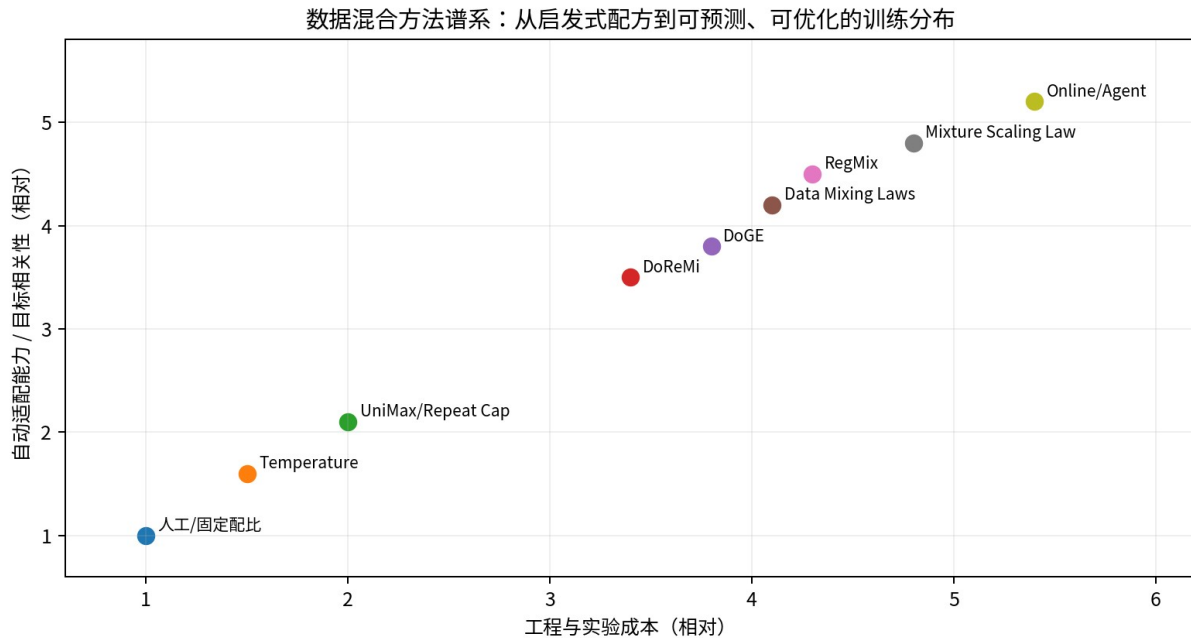


图3 方法谱系（概念图）：自动化越强，通常越依赖 proxy experiments 与可靠 eval。

8. RegMix：把 Mixture Search 变成回归问题

RegMix 训练许多小模型覆盖不同 mixture，用 regression model 预测未训练 mixture 的性能，再将预测最好的 mixture 用于大模型训练。论文用 512 个 1M 参数、各训练 1B tokens 的小模型拟合回归器，预测出的 mixture 用于 1B 参数/25B token 训练，并在 64 个候选大模型配方中表现最好。[9]

作者还报告：在最高 7B、100B tokens 的实验中，RegMix 持续优于人工选择，并以约 DoReMi 10% 的计算开销达到或超过其结果。[9]

RegMix 实验设计	Know-how
Mixture design space	不要只沿单轴扫比例；要覆盖 simplex，才能学到域交互
Tiny models	模型必须便宜到可以训练数百个；目标是学 response surface
Labels	可用 validation loss、综合 benchmark 或多目标标量

RegMix 实验设计	Know-how
Regressor	重点不是模型多复杂，而是能否对 unseen mixture 排序正确
Search	在 surrogate 上搜索，比在大模型上网格搜索便宜几个数量级
Validation	至少保留若干大模型候选 A/B，避免 surrogate extrapolation 失真

Know-how RegMix 的工程价值在于把昂贵的大模型试错，转成“数据生成（小训练）→ surrogate fitting → search”。Mixture optimization 因而开始像传统系统优化/AutoML，而不再只是训练负责人手调百分比。

9. Data Mixing Laws：配方效果具有可预测结构

Data Mixing Laws 发现语言模型性能相对于 mixture proportions 可以用函数形式拟合；通过少量 sample mixtures，可以预测 unseen mixtures，并把 mixture law 与 training-step/model-size scaling law 嵌套，用小规模实验预测大规模训练。[8]

其 RedPajama 实验中，1B 模型用优化 mixture 训练 100B tokens，达到接近默认 mixture 多训练 48% steps 的性能；在 continual training 中还能预测避免 catastrophic forgetting 的临界 mixture proportion。[8]

2025 的 Scaling Laws for Optimal Data Mixtures 进一步把 loss 建模为 model size N 、token budget D 与 domain weight vector h 的函数，并报告可从少量小规模 run 外推到 unseen domain weights 与更大 scale。[11]

问题	传统做法	Mixing/Scaling Law 做法
选 mixture	大模型多轮试错	小模型点采样 + 函数拟合
换模型规模	重新调配方	将 N 纳入预测变量
换 token horizon	沿用旧权重	将 D / training steps 纳入预测
目标域切换	人工再调	对目标 validation loss 求最优 h
预算规划	拍脑袋 reserve	先预测各 mixture 的边际收益，再分配 proxy budget

10. Static Mixture → Curriculum：为什么 $\alpha(t)$ 往往比 α 更合理

静态权重假设“训练第 1T token 和第 10T token 的数据价值相同”。现实中并非如此：模型已学会的域会出现边际递减，稀缺数据会接近重复上限，后期高质量/目标域数据可能更有价值。

NVIDIA 的 Two-Phase Pretraining 系统研究将 selection、blending 和 ordering 统一考虑。其两阶段方案相对随机数据排序与 natural token distribution 分别报告约 3.4% 和 17% 的平均准确率改进，并将 1T-token 小规模 blend 设计外推到 15T tokens、25B 模型。[10]

Upsample or Upweight? 进一步指出：在 SGD 中，temperature upsampling 与 loss upweighting 并不等价；temperature sampling 收敛更快但更容易过拟合尾部数据，作者据此提出训练过程中降低采样温度的 Cooldown。[13]

阶段	常见目标	可能的 mixture 变化
Early	建立通用语言/世界表示	高 diversity、大量 unique Web/Books
Middle	补 code/math/science 等结构能力	提高目标能力域但控制 epochs
Late / Anneal	提升高质量与目标任务敏感能力	增加 quality buckets、reasoning/code；降低噪声
Continual PT	加入新领域且防遗忘	target + source replay，寻找临界比例

阶段	常见目标	可能的 mixture 变化
Long-context phase	适应长序列	长文档 + 高质量短文混合，而非只喂长文本

核心判断 Mixture 是 schedule，不只是 vector。生产系统最好保存 $\alpha(t)$ 或 phase-wise mixture，而不是只在模型卡里写一张“总体占比”表。

11. 多语言、Code/Math 与质量分桶：Mixture 可以是多层层级结构

真实系统通常不是在一张平面表上混 1000 个 source，而是 hierarchical mixture：先分 modality/domain，再在每个域中分 language、quality bucket、source、time slice、document length。

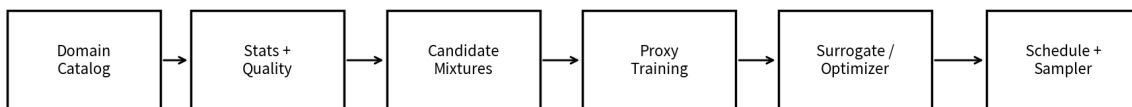
示例：Domain $\rightarrow$ {Web 60%, Code 20%, Math 10%, Books 10%}; Web 内再按 {English, Chinese, Spanish,...} temperature sampling; English Web 内再按 quality buckets 进行 soft sampling。

层级	控制项	典型约束
Domain	Web/Code/Math/Books	能力目标、跨域迁移
Language	en/zh/...	temperature、fairness、tail repeat cap
Quality	Q0-Q5	高分 upweight，但保留 diversity floor
Source	CC/FineWeb/Wiki/...	source quota、防单一 pipeline bias
Time	crawl month/year	新鲜度 vs 稳定知识
Length	short/medium/long	context skill 与 packing 效率
License/Governance	allowed/restricted tags	硬约束不应由 utility optimizer 覆盖

Know-how 把“硬约束”和“效用优化”分层：license、安全、隐私等 policy gate 先决定可用集合；mixture optimizer 只能在 allowed pool 上优化，不能用性能收益覆盖治理约束。

12. 生产级 Mixture Optimizer Blueprint

推荐生产架构：Mixture 是“可实验、可回放、可审计”的训练策略



输出不是一个裸比例，而是 versioned mixture policy：domain weights + repeat caps + phase schedule + seed + sampler semantics

图 4 生产级 mixture pipeline：以可回放的 policy artifact 作为最终产物。

模块	输入	输出	关键指标
Domain Catalog	dataset metadata / token counts	domain taxonomy	unique tokens、lang、quality、license

模块	输入	输出	关键指标
Mixture Generator	bounds / priors	candidate α vectors	simplex coverage、repeat constraints
Proxy Lab	candidates	loss/eval curves	fixed-compute comparability
Surrogate	proxy results	performance predictor	rank correlation、OOD mixture error
Optimizer	predictor + objectives	best/Pareto mixtures	target utility、robustness
Scheduler	weights + phases	$\alpha(t)$	phase boundaries、anneal
Sampler	$\alpha(t)$ +shards+seed	training batches	realized vs planned share
Observability	runtime logs	mixture audit	effective epochs、drift、starvation

13. Mixture 实验：最小可执行协议

如果没有大规模算力，最值得复制的是“多小实验 + 少量大验证”的结构，而不是直接在目标模型上改百分比。

Step	具体做法	为什么
1. 定域	每个 domain 必须语义清楚、互斥/可解释	垃圾 taxonomy 会让 optimizer 学到错误控制变量
2. 统计	token、unique token、quality、epochs upper bound	先知道 supply constraint
3. 建 baseline	natural + temperature + human recipe	没有强 baseline 就无法判断自动方法价值
4. 采 candidates	Dirichlet / Latin hypercube / simplex grid	覆盖域间交互，不只做 one-factor-at-a-time
5. 训练 proxy	固定 model/steps/batch/seed family	确保 performance difference 来自 mixture
6. 多目标 eval	per-domain val loss + general benchmark + target tasks	避免只为单个 benchmark hill-climb
7. Fit/search	RegMix / mixing law / Bayesian surrogate	在低成本空间预测未见配方
8. Stress test	不同 scale、token horizon、seeds	检查 proxy transfer 与排名稳定性
9. Large A/B	至少 baseline / human / optimized	最终 ground truth 是目标规模训练
10. Freeze artifact	记录 weights、caps、schedule、hash、seed	让模型训练可重现可审计

14. 最常见失败模式

失败模式	症状	根因	修复
只看 raw proportion	小域被重复几十 epoch	没计算 effective epochs	每域输出 sampled/unique token 比
只加“高质量”	短期 eval 好、长训变差	unique tokens 不足	质量 bucket + diversity floor + long-horizon ablation
直觉认为 Wiki/ArXiv 必须高权重	自动优化反而大量降权	忽略跨域迁移与 sample efficiency	以 proxy results 而非声誉决定
小模型最优直接外推	大模型排名翻转	scale-dependent utility	至少 2-3 个 proxy scales
把 upsample 当 upweight	收敛/过拟合行为不同	SGD 下二者不等价	显式选择 sampler vs loss weighting
优化 benchmark 单指标	其他能力退化	objective 太窄	Pareto / multi-domain validation
不记录 sampler 实现	同权重跑出不同实际分布	shard/seed/packing 语义不同	记录 realized mixture 与 RNG state
持续预训练 100% 新域	通用能力快速下降	distribution shift / forgetting	source replay + critical mixture search

15. 2026 前沿：从“选一个配方”走向“受约束的最优控制”

2025 的 Scaling Laws for Optimal Data Mixtures 把最优 domain weights 与模型规模 N 、token budget D 联合建模；2026 的 mixture-under-constraints 工作又显式把目标域重复的边际递减与通用数据的 regularization 纳入 scaling law。[11][12]

2025 的 Data Mixing Agent 则进一步尝试用强化学习从大量 mixture trajectories 中学习可迁移的 reweighting heuristic，在 continual pretraining 的 math/code 场景中自动平衡目标域收益与 source capability preservation。该方向较新，证据成熟度低于 DoReMi/RegMix/Scaling-Law 系列，应视为前沿探索。[14]

趋势判断 数据混合正在从“经验配方”演化成一个受约束控制问题：状态 = 模型学习进度与数据供给；动作 = domain weights / repeat caps；奖励 = 多目标能力增益；约束 = compute、unique tokens、治理与遗忘。真正成熟的系统会把 mixture optimizer 与训练 runtime、eval loop 连接起来。

16. Know-Why / Know-How 沉淀

原则	Know-why	Know-how
Mixture 是 exposure	compute 花在被采到的 token 上	所有策略统一转换成 sampled tokens + effective epochs
高质量 ≠ 高权重	重复边际递减、跨域迁移存在	质量 score 进入 mixture optimizer，而不是直接乘权重
小数据可以重复	混合训练有 regularization	设 repeat cap，做 horizon-dependent ablation
Web 不只是噪声	覆盖广、提供共享表示基础	不要因“看起来普通”而过度压低 Web
域之间有交互	Code/Math/Web/Books 不独立	candidate design 要覆盖 interaction
Proxy 是实验加速器	大模型网格搜索不可承受	DoReMi/RegMix/Mixing Laws 都利用 small-to-large transfer
静态比例不够	不同阶段边际效用变化	用 phase-wise $\alpha(t)$ + annealing
最优配方不是普适常数	依赖 N、D、目标、corpus supply	把模型规模和训练 horizon 纳入 mixture search
最终证据是大训练	surrogate 可能外推错误	small-run search + large-scale confirm
可重放比漂亮数字重要	sampler 细节会改变实际 exposure	版本化 mixture policy、seed、shard、realized stats

17. 一页 Production Checklist

- 定义 domain taxonomy，避免同一文档在多个域重复计权。
- 记录每个域 unique tokens、tokenizer 后规模、quality buckets、license/policy tags。
- 给出 Natural / Temperature / Human baseline。
- 每个 candidate 都计算 expected epochs 与最大重复次数。
- 多语言至少同时报告 raw share、sampled share、repeat cap。
- 优先做几十到数百个 cheap proxy runs，而不是少数昂贵大模型盲试。
- Mixture objective 同时包含 per-domain validation loss 与核心 downstream eval。
- 检查 proxy ranking 在不同 model size / token horizon / seed 上是否稳定。
- 区分 upsampling 与 loss upweighting；不要默认二者等价。
- 长 horizon 配方必须检查 unique-token supply 与重复边际递减。
- 持续预训练必须保留 source replay，并搜索防遗忘的临界比例。
- 所有 mixture 都以 versioned artifact 保存：weights、phase、caps、seed、sampler code hash。
- 训练时实时对比 planned vs realized mixture，监控域饥饿、过采样和 shard drift。
- 最终大模型至少验证 baseline / human recipe / optimized recipe 三组。

一句话总结 Data Mixture 的核心不是“找出正确百分比”，而是建立一个低成本、可预测、可迁移的实验系统，让每一单位训练 compute 被分配给边际训练价值更高的 token，同时不牺牲多样性、稀缺域、治理约束和长期训练稳定性。

参考资料与证据等级

资料	本册使用方式
[1] The Pile: An 800GB Dataset of Diverse Text for Language Modeling. Gao et al., 2020. arXiv:2101.00027.	基础数据混合/域化语料；用于背景。
[2] LLaMA: Open and Efficient Foundation Language Models. Touvron et al., 2023. arXiv:2302.13971.	一手模型论文；公开具体 sampling proportions 与 epochs。
[3] mT5: A Massively Multilingual Pre-trained Text-to-Text Transformer. Xue et al., 2020/NAACL 2021. arXiv:2010.11934.	一手多语言采样；temperature $\alpha=0.3$ 。
[4] UniMax: Fairer and more Effective Language Sampling for Large-Scale Multilingual Pretraining. Chung et al., 2023. arXiv:2304.09151.	一手方法；重复上限/多语言 sampling。
[5] Scaling Data-Constrained Language Models. Muennighoff et al., 2023. arXiv:2305.16264.	400-run 系列；重复数据与 compute scaling。
[6] DoReMi: Optimizing Data Mixtures Speeds Up Language Model Pretraining. Xie et al., 2023. arXiv:2305.10429.	核心一手方法；proxy + Group DRO + excess loss。
[7] DoGE: Domain Reweighting with Generalization Estimation. Fan et al., 2023/2024. arXiv:2310.15393.	核心一手方法；bi-level generalization optimization。
[8] Data Mixing Laws: Optimizing Data Mixtures by Predicting Language Modeling Performance. Ye et al., ICLR 2025. arXiv:2403.16952.	混合比例可预测性与规模外推。
[9] RegMix: Data Mixture as Regression for Language Model Pre-training. Liu et al., ICLR 2025. arXiv:2407.01492.	512 tiny-model mixture search；surrogate regression。
[10] Maximize Your Data's Potential: Enhancing LLM Accuracy with Two-Phase Pretraining. Feng et al., 2024. arXiv:2412.15285.	两阶段 blend/order；1T→15T/25B scale study。
[11] Scaling Laws for Optimal Data Mixtures. Shukor et al., 2025. arXiv:2507.09404.	较新预印本；N/D/mixture 联合 scaling law。
[12] Scaling Laws for Mixture Pretraining Under Data Constraints. Sedova et al., 2026. arXiv:2605.12715.	最新预印本；2,000+ runs、repetition-aware mixture law。
[13] Upsample or Upweight? Balanced Training on Heavily Imbalanced Datasets. Li et al., 2024. arXiv:2410.04579.	理论+实验；SGD 下 upsampling 与 upweighting 差异。
[14] Data Mixing Agent: Learning to Re-weight Domains for Continual Pre-training. Yang et al., 2025. arXiv:2507.15640.	前沿预印本；RL agent 自动学习 reweighting heuristic。

证据分层：DoReMi、RegMix、UniMax、Data Mixing Laws 等已有较完整实验/同行评审或成熟复现背景；2025-2026 的 scaling-law / agent 方向属于更新研究，本册将其作为趋势与可验证假设，而不是已定工程常识。

检索与整理日期：2026-08-13。本文中的“示意图”均为机制解释图，不冒充论文原始实验曲线；论文数值均在正文标注引用编号。